A trace semantics for the process calculus of ProVerif

Artie Khovanov with lots of assistance from Claude Fable

ProVerif is a tool for reasoning about cryptographic security in a symbolic model with a Dolev-Yao adversary. Although it uses resolution of Horn clauses to find attacks, it is backed theoretically by a process calculus, which is a variant of the applied pi calculus. The ground truth for what ProVerif code means is the formal semantics of its input language in Appendix B of the ProVerif manual [2], which takes precedence over the manual's informal chapters and over the tool itself: ProVerif's resolution procedure, and the encodings it applies to a model beforehand, approximate that semantics and are not consulted. Where the manual refers to a paper for a component, that paper is the ground truth for it: the adversary and the security properties are those of Blanchet's survey [1], to which the manual refers for them, extended to the manual's terms with effects. The attacker is the abstract one.

We formalise the ProVerif process calculus in Lean. Rather than copying the specification, we use a shallow translation in order to make security proofs simpler down the line. This means that we make use of existing Lean syntax and constructs where possible. In particular, some invariants are enforced only by what terms can arise from translating a valid ProVerif program. We also deviate from the calculus when sticking exactly to the specification would complicate the design.

We use a trace-based semantics to encode the possible outcomes of parallel processes communicating on a network. The attacker is modelled as a set of traces satisfying informational constraints. Our trace semantics are good enough to express secrecy and simple ProVerif queries relating events. However, it is strictly less expressive than the formal operational semantics of ProVerif; in particular, we cannot express observational equivalence.

We give a mathematical account of the Lean development, and of the formal translation from terms in ProVerif's process calculus to our traces. Everything has been formalised in Lean.

Section 1 presents the trace semantics; Section 2 the calculus of ProVerif, its syntax and the semantics of the manual; and Section 3 the translation of the calculus into the trace semantics and the theorems relating the two. In the development these are the folders `TraceSemantics` and `Specification`, the translation being written twice: as the denotation in `Specification`, which the theorems are about, and as the program `Translation`, which turns a ProVerif model into Lean source over the trace semantics.

1 The trace semantics

1.1 Messages

Definition 1.1 (Signature, terms)**.** A *signature* is a family $\Sigma = (\Sigma_n)_{n\in\mathbb{N}}$ of sets; $f \in \Sigma_n$ is a constructor of arity n. The set $\mathcal{T} = \mathcal{T}(\Sigma)$ of *raw terms* is generated by

$$t ::= \nu_k \ (k \in \mathbb{N}) \ \mid \ f(t_1, \dots, t_n) \ (f \in \Sigma_n).$$

A *substitution* is a map $\sigma : \mathbb{N} \to \mathcal{T}$; $t\sigma$ replaces each ν_k in t by $\sigma(k)$.

Definition 1.2 (Theory)**.** A *theory* over Σ is a pair (E, Fr) with $E \subseteq \mathcal{T} \times \mathcal{T}$ and $\mathrm{Fr} \subseteq \bigcup_n \Sigma_n$ such that

(H) for every $(l, r) \in E$, both l and r are of the form $f(\dots)$ with $f \notin \mathrm{Fr}$.

The relation $\equiv_E$ is the least congruence on $\mathcal{T}$ containing $l\sigma \equiv_E r\sigma$ for all $(l, r) \in E$ and all substitutions σ. The set of *messages* is $\mathcal{M} = \mathcal{T}/\!\equiv_E$, with $[t]$ the class of t. Constructors act on $\mathcal{M}$ by $f([t_1], \dots, [t_n]) = [f(t_1, \dots, t_n)]$, and ν_k also denotes $[\nu_k]$.

A Σ-algebra A *satisfies* E if $l\sigma_A = r\sigma_A$ for every $(l, r) \in E$ and every valuation $\sigma_A : \mathbb{N} \to A$, where $t\sigma_A$ is the evaluation of t in A.

Proposition 1.3. *$\mathcal{M}$ satisfies E. For every Σ-algebra A satisfying E and every $v : \mathbb{N} \to A$ there is a unique homomorphism $\mathcal{M} \to A$ sending ν_k to $v(k)$.*

Definition 1.4 (View)**.** Let $\mathcal{V} = \mathbb{N} \uplus \{f(m_1, \dots, m_n) \mid f \in \mathrm{Fr} \cap \Sigma_n,\ m_i \in \mathcal{M}\} \uplus \{\bot\}$. Define $\mathrm{view}_0 : \mathcal{T} \to \mathcal{V}$ by

$$\mathrm{view}_0(\nu_k) = k, \qquad \mathrm{view}_0(f(t_1, \dots, t_n)) = \begin{cases} f([t_1], \dots, [t_n]) & f \in \mathrm{Fr}, \\ \bot & f \notin \mathrm{Fr}. \end{cases}$$

Proposition 1.5. *The following hold.*

1. *$s \equiv_E t$ implies $\mathrm{view}_0(s) = \mathrm{view}_0(t)$; hence view_0 induces $\mathrm{view} : \mathcal{M} \to \mathcal{V}$.*
2. *$\mathrm{view}(f(m_1, \dots, m_n)) = f(m_1, \dots, m_n)$ if $f \in \mathrm{Fr}$, and $\bot$ otherwise.*
3. *If $\mathrm{view}(m) = f(m_1, \dots, m_n)$ then $m = f(m_1, \dots, m_n)$.*
4. *$k \mapsto \nu_k$ is injective $\mathbb{N} \to \mathcal{M}$.*

Definition 1.6 (May-fail messages, lifting)**.** Let $\mathcal{M}_\bot = \mathcal{M} \uplus \{\bot\}$. For $g : \mathcal{M}^n \to \mathcal{M}_\bot$ let $g^\uparrow : \mathcal{M}_\bot^n \to \mathcal{M}_\bot$ be $g^\uparrow(\vec{u}) = g(\vec{u})$ if every $u_i \in \mathcal{M}$ and $\bot$ otherwise; $f^\uparrow$ for $f \in \Sigma_n$ is the lift of $f : \mathcal{M}^n \to \mathcal{M} \subseteq \mathcal{M}_\bot$. A *destructor* of arity n is a function $\mathcal{M}_\bot^n \to \mathcal{M}_\bot$.

Definition 1.7 (Data symbols, standard operations)**.** The *data symbols* are a tuple constructor tuple_n of each arity $n \in \mathbb{N}$ and the constants true and false. A signature *with data* is $\Sigma = \Sigma_0 \uplus \mathrm{Data}$ for a signature Σ_0 of *declared* symbols, and a theory (E, Fr) over Σ_0 is read over Σ with the data symbols free. We write $(m_1, \dots, m_n)$ for $\mathsf{tuple}_n(m_1, \dots, m_n)$. The *standard operations* Std are the lifts $\mathsf{tuple}_n^\uparrow$, the projections $\pi_i^n : \mathcal{M}_\bot \to \mathcal{M}_\bot$ with $\pi_i^n(u) = m_i$ if $u = (m_1, \dots, m_n)$ and $\bot$ otherwise, well defined by Proposition 1.5, and the constants true and false.

Definition 1.8 (Public operations, deduction)**.** Let $\mathrm{Pub} \subseteq \bigcup_n \Sigma_n$ and let Dt be a set of destructors. Set $\mathrm{Ops} = \{f^{\uparrow} \mid f \in \mathrm{Pub}\} \cup \mathrm{Dt} \cup \mathrm{Std}$. A set $S \subseteq \mathcal{M}$ is Ops-*closed* if for every $o \in \mathrm{Ops}$ of arity n and every $\vec{u} \in (S \uplus \{\bot\})^n$, $o(\vec{u}) \in \mathcal{M}$ implies $o(\vec{u}) \in S$. For $K \subseteq \mathcal{M}$,

$$\langle K \rangle = \bigcap \{S \supseteq K \mid S \text{ is Ops-closed}\}, \qquad K \vdash m \iff m \in \langle K \rangle.$$

Proposition 1.9. *$\langle\cdot\rangle$ is a closure operator on $\mathcal{P}(\mathcal{M})$, $\langle K \rangle$ is* Ops*-closed, and $K \vdash m_1, \ldots, K \vdash m_n$ imply $K \vdash f(m_1, \ldots, m_n)$ for $f \in \mathrm{Pub} \cap \Sigma_n$ or f a data symbol. In particular $\emptyset \vdash a$ for $a \in \mathrm{Pub} \cap \Sigma_0$.*

Remark 1.10. In the development the data symbols are added to a signature and a theory by withData, and the standard operations are the class StdOps on a type of messages, which the messages of a theory with data instantiate; Ops is the class Attacker, and Attacker.ofDecls builds it from Pub and Dt.

1.2 Traces and processes

Definition 1.11 (Actions, traces)**.**

$$\mathrm{Act} = \{\mathsf{out}(c,m),\ \mathsf{in}(c,m),\ \mathsf{comm}(c,m),\ \mathsf{new}(k),\ \mathsf{ev}(m) \mid c, m \in \mathcal{M},\ k \in \mathbb{N}\}.$$

A *trace* is a word $t \in \mathrm{Act}^*$; ε is the empty trace and $l \leq t$ means l is a prefix of t. The actions $\mathsf{out}(c,m)$ and $\mathsf{in}(c,m)$ are *unfinished*; $\mathrm{Closed}(t)$ holds if no action of t is unfinished. $\mathrm{nonces}(t) \in \mathbb{N}^*$ is the image of t under the monoid homomorphism with $\mathsf{new}(k) \mapsto k$ and every other action $\mapsto \varepsilon$. A *process* is a subset of Act^*; $\mathrm{Proc} = \mathcal{P}(\mathrm{Act}^*)$.

Definition 1.12 (Sequential constructors)**.** For $a \in \mathrm{Act}$, $P \in \mathrm{Proc}$, $F : \mathcal{M} \to \mathrm{Proc}$:

$$\begin{aligned} a \cdot P &= \{\varepsilon\} \cup \{a\,t \mid t \in P\}, & \mathbf{0} &= \{\varepsilon\}, \\ \mathsf{out}(c,m).P &= \mathsf{out}(c,m) \cdot P, & \mathsf{ev}(m).P &= \mathsf{ev}(m) \cdot P, \\ \mathsf{in}(c).F &= \textstyle\bigcup_{m \in \mathcal{M}} \mathsf{in}(c,m) \cdot F(m), & \mathsf{new}.F &= \textstyle\bigcup_{k \in \mathbb{N}} \mathsf{new}(k) \cdot \{t \in F(\nu_k) \mid k \notin \mathrm{nonces}(t)\}. \end{aligned}$$

Definition 1.13 (Parallel composition)**.** Let I be a set. For $s \in (I \times \mathrm{Act})^*$ and $i \in I$, $s\restriction i \in \mathrm{Act}^*$ is the word of actions of s tagged i, in order, and $\overline{s} \in \mathrm{Act}^*$ is s with tags removed. For $f : I \to \mathrm{Act}^*$, $\mathrm{Interleaves}(f,s)$ holds if $s\restriction i = f(i)$ for all i. The relation $\mathrm{comms} \subseteq (I \times \mathrm{Act})^* \times \mathrm{Act}^*$ is the least one such that

$$\begin{aligned} &\mathrm{comms}(\varepsilon, \varepsilon), \\ &\mathrm{comms}(s,t) \Rightarrow \mathrm{comms}\big((i,a)\,s,\ a\,t\big), \\ &\mathrm{comms}(s,t) \wedge i \neq j \Rightarrow \mathrm{comms}\big((i,\mathsf{out}(c,m))\,(j,\mathsf{in}(c,m))\,s,\ \mathsf{comm}(c,m)\,t\big). \end{aligned}$$

For $P : I \to \mathrm{Proc}$,

$$\begin{aligned} \mathrm{merge}_I(P) = \{t \mid \exists f, s.\ (\forall i.\ f(i) \in P(i)) \ \wedge\ &\mathrm{Interleaves}(f,s) \\ &\wedge\ \mathrm{comms}(s,t)\ \wedge\ \mathrm{nonces}(t) \text{ has no repeated letter}\}. \end{aligned}$$

$P \mid Q = \mathrm{merge}_{\{0,1\}}(0 \mapsto P,\ 1 \mapsto Q)$ and $!P = \mathrm{merge}_{\mathbb{N}}(k \mapsto P)$.

Proposition 1.14. *Let* $\mathrm{comms}(s,t)$.

1. $\mathrm{nonces}(t) = \mathrm{nonces}(\overline{s})$.
2. *If* a *occurs in* t *then* (i,a) *occurs in* s *for some* i*, or* $a = \mathsf{comm}(c,m)$ *and* $(i,\mathsf{out}(c,m))$, $(j,\mathsf{in}(c,m))$ *occur in* s *for some* $i \neq j$.
3. *If* $\mathrm{Closed}(t)$ *and* $(k,\mathsf{in}(c,m))$ *occurs in* s*, then* $(i,\mathsf{out}(c,m))$ *occurs in* s *for some* $i \neq k$.
4. $!P = P \mid !P$ *for every* $P \in \mathrm{Proc}$.

1.3 The attacker and the security properties

Definition 1.15 (Enemy)**.** The processes $\mathrm{Enemy}(K) \in \mathrm{Proc}$, for $K \subseteq \mathcal{M}$, are the pointwise least such that

1. $\varepsilon \in \mathrm{Enemy}(K)$;
2. if $K \vdash c$ and $e \in \mathrm{Enemy}(K \cup \{m\})$ then $\mathsf{in}(c,m)\, e \in \mathrm{Enemy}(K)$, for every $m \in \mathcal{M}$;
3. if $K \vdash c$, $K \vdash m$ and $e \in \mathrm{Enemy}(K)$ then $\mathsf{out}(c,m)\, e \in \mathrm{Enemy}(K)$;
4. if $K \vdash c$, $K \vdash m$ and $e \in \mathrm{Enemy}(K)$ then $\mathsf{comm}(c,m)\, e \in \mathrm{Enemy}(K)$, an output to itself;
5. if $k \notin \mathrm{nonces}(e)$ and $e \in \mathrm{Enemy}(K \cup \{\nu_k\})$ then $\mathsf{new}(k)\, e \in \mathrm{Enemy}(K)$.

Definition 1.16 (Attacks, knowledge)**.** For $P \in \mathrm{Proc}$,

$$\mathrm{Attacks}(P) = \{t \in P \mid \mathrm{Enemy}(\emptyset) \ : \ \mathrm{Closed}(t)\}.$$

For $K \subseteq \mathcal{M}$, $\mathrm{known}(K,\cdot) : \mathrm{Act}^* \to \mathcal{P}(\mathcal{M})$ is defined by

$$\begin{aligned} \mathrm{known}(K,\varepsilon) &= K, \\ \mathrm{known}(K,a\,t) &= \mathrm{known}\big(\langle K \cup \{m \mid a = \mathsf{comm}(c,m),\ c \in K\}\rangle,\, t\big). \end{aligned}$$

Proposition 1.17 (Prefix-closure)**.** $\mathrm{Enemy}(K)$ *and every process built from the constructors of Definitions 1.12 and 1.13 are prefix-closed. If* P *is prefix-closed, so is* $\mathrm{Attacks}(P)$.

Definition 1.18 (Facts, secrecy, correspondence)**.** A *fact* is $\mathsf{event}(e)$ or $\mathsf{knows}(m)$ with $e, m \in \mathcal{M}$; Fact is the set of facts. A fact *holds* after $t \in \mathrm{Act}^*$, written $t \models f$, if

$$t \models \mathsf{event}(e) \iff \mathsf{ev}(e) \in t, \qquad t \models \mathsf{knows}(m) \iff m \in \mathrm{known}(\langle\emptyset\rangle, t).$$

For a set X, $F, H : (X \to \mathcal{M}) \to \mathrm{Fact}$ and $s \in \mathcal{M}$,

$$\begin{aligned} \mathrm{Corr}(P,F,H) \iff{}& \forall \sigma : X \to \mathcal{M}.\ \forall t \in \mathrm{Attacks}(P). \\ & t \models F(\sigma) \Rightarrow \exists \sigma' : X \to \mathcal{M}.\ F(\sigma') = F(\sigma) \wedge t \models H(\sigma'), \\ \mathrm{Secret}(P,s) \iff{}& \forall t \in \mathrm{Attacks}(P).\ t \not\models \mathsf{knows}(s). \end{aligned}$$

By Proposition 1.17, for prefix-closed P these are properties of every prefix of every attack: $\mathrm{Corr}(P,F,H)$ says that some $H(\sigma')$ holds by the time $F(\sigma)$ does.

2 The calculus of ProVerif

This section gives the syntax of the fragment of ProVerif's input language that the development covers and restates its semantics from the ProVerif manual (Appendix B). The manual describes the attacker and the properties its queries test informally (§3.2, §4.3.1) and refers to the survey for their definitions (Chapter 7), so the adversary, secrecy and correspondences are the survey's (§3.1–3.3), stated here for the manual's semantics.

2.1 Syntax

Definition 2.1 (Source syntax)**.** Fix a set of variables $x, y, \ldots$ and a set of names $a, b, c, \ldots$ containing the free names $\mathcal{N}_{\text{pub}} \uplus \mathcal{N}_{\text{priv}}$. Terms, expressions and processes are generated by

$$\begin{aligned} M, N &::= x \mid a \mid f(M_1, \ldots, M_n), \\ D &::= M \mid h(D_1, \ldots, D_n) \mid \mathsf{fail} \mid \mathsf{new}\ a; D \mid \mathsf{if}\ D\ \mathsf{then}\ D_1\ \mathsf{else}\ D_2 \\ &\quad \mid \mathsf{let}\ x = D\ \mathsf{in}\ D_1\ \mathsf{else}\ D_2 \mid \mathsf{event}\ ev(D_1, \ldots, D_n); D', \\ P, Q &::= \mathbf{0} \mid \mathsf{out}(D, D'); P \mid \mathsf{in}(D, x); P \mid P \mid Q \mid !P \mid \mathsf{new}\ a; P \\ &\quad \mid \mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q \mid \mathsf{if}\ D\ \mathsf{then}\ P\ \mathsf{else}\ Q \mid \mathsf{event}\ ev(D_1, \ldots, D_n); P, \end{aligned}$$

where f ranges over constructors, h over constructors and destructors and ev over event symbols. A term is an expression, and a *may-fail term* is $U ::= M \mid \mathsf{fail}$. An *event* e is $ev(M_1, \ldots, M_n)$ with ev an event symbol of arity n and $M_1, \ldots, M_n$ terms; it is not a term, and $=_E$ and substitution extend to events componentwise. Types are erased, and with them the [typeConverter] constructors, which are the identity when types are ignored (§4.1.2). These are the manual's terms, process terms and processes (Appendix A, Figures A.1 and A.8) without patterns other than variables, tables, predicates, phases, synchronisation and natural numbers. The manual's $x \leftarrow D; D'$ and $x \leftarrow D; P$ abbreviate $\mathsf{let}\ x = D\ \mathsf{in}\ D'\ \mathsf{else}\ \mathsf{fail}$ and $\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ \mathbf{0}$, its $a \xleftarrow{R} t$ abbreviates $\mathsf{new}\ a$, and an omitted else is $\mathsf{else}\ \mathsf{fail}$ in an expression and $\mathsf{else}\ \mathbf{0}$ in a process. An application $g(D_1, \ldots, D_n)$ of a function macro $\mathsf{letfun}\ g(x_1, \ldots, x_n) = D$ (§4.2.3), none of whose parameters is declared $\mathsf{or}\ \mathsf{fail}$, abbreviates $\mathsf{let}\ x_1 = D_1\ \mathsf{in}\ \ldots\ \mathsf{let}\ x_n = D_n\ \mathsf{in}\ D$ with every else omitted, and a call $p(D_1, \ldots, D_n)$ of a process macro $\mathsf{let}\ p(x_1, \ldots, x_n) = P$ (§3.1.3, §4.2.4), none of whose parameters is declared $\mathsf{or}\ \mathsf{fail}$, likewise abbreviates $\mathsf{let}\ x_1 = D_1\ \mathsf{in}\ \ldots\ \mathsf{let}\ x_n = D_n\ \mathsf{in}\ P$ with every else omitted.

Definition 2.2 (Rewrite rules)**.** Fix a set of *may-fail variables* u, disjoint from the variables of Definition 2.1, which are the *ordinary variables*. The may-fail terms of a rule are $U ::= M \mid u \mid \mathsf{fail}$ with M a term without names: the may-fail terms of Definition 2.1 with may-fail variables besides. Each destructor g of arity n carries a finite ordered list $\mathrm{def}(g)$ of *rewrite rules* $g(U_1, \ldots, U_n) \to U$ between may-fail terms, the variables of U occurring in $U_1, \ldots, U_n$, ending with $g(\vec{u}) \to \mathsf{fail}$ for distinct may-fail variables $\vec{u}$. A *substitution* maps ordinary variables to terms and may-fail variables to may-fail terms. The manual writes $\mathrm{def}(g)$ as a set of rules each with a side condition that excludes the rules before it (Appendix B); ProVerif generates that form from the ordered list, and the two define the same rewriting.

Definition 2.3 (Model)**.** A *model* consists of finite disjoint sets $\mathcal{N}_{\text{pub}}$ and $\mathcal{N}_{\text{priv}}$ of free names; a finite set of *declared* function symbols, each with an arity, each a constructor or a destructor, and each public or private, the destructors with their rules $\mathrm{def}(g)$ as in Definition 2.2; a finite set of *event symbols*, each with an arity; an *equational theory* E, a finite set of equations $M = N$

between terms over the constructors without names; and a closed process P_0 with $\mathrm{fn}(P_0) \subseteq \mathcal{N}_{\mathrm{pub}} \uplus \mathcal{N}_{\mathrm{priv}}$. The constructors of the calculus are the declared ones and the *data symbols*, a tuple constructor $(\cdot, \ldots, \cdot)$ of each arity and the constants true and false; its destructors are the declared ones and the *built-in* ones, the projections $i\mathrm{th}_n$, each with the single rule $i\mathrm{th}_n((x_1, \ldots, x_n)) \to x_i$, and the boolean destructors equal, not, and and or with the rules

$$\begin{array}{llll} \mathsf{equal}(x,x) \to \mathsf{true}, & \mathsf{equal}(x,y) \to \mathsf{false}, & \mathsf{not}(\mathsf{true}) \to \mathsf{false}, & \mathsf{not}(x) \to \mathsf{true}, \\ \mathsf{and}(\mathsf{true},u) \to u, & \mathsf{and}(x,u) \to \mathsf{false}, & \mathsf{or}(\mathsf{true},u) \to \mathsf{true}, & \mathsf{or}(x,u) \to u, \end{array}$$

u a may-fail variable, so that $\mathsf{and}(\mathsf{false}, \mathsf{fail})$ rewrites to false; $M = N$, $M \neq N$, $M \mathbin{\&\&} N$, $M \mid\mid N$ and $\mathsf{not}(M)$ abbreviate the expressions $\mathsf{equal}(M,N)$, $\mathsf{not}(\mathsf{equal}(M,N))$, $\mathsf{and}(M,N)$, $\mathsf{or}(M,N)$ and $\mathsf{not}(M)$. The rules of every destructor are over the constructors of the calculus. The data symbols and the built-in destructors are public. These are the survey's tuples, projections and booleans (§2.1, (2.5)) and the manual's infix operators (Appendix B), built in as ProVerif has them.

Definition 2.4 (Queries)**.** A *query fact* is $\mathsf{event}(e)$ or $\mathsf{attacker}(M)$ with e an event and M a term, both over constructors, free names and a finite set X of variables; it is an *event fact* or an *attacker fact* accordingly. A *query* is $\mathsf{query}\ \mathsf{attacker}(M)$ with M without variables, or $\mathsf{query}\ F \Longrightarrow H$ with F and H query facts over X.

2.2 Semantics

Names are atoms. The equational theory E of the model gives $=_E$, the closure of E under reflexivity, symmetry, transitivity, application of function symbols and substitution. The manual states its semantics for the full input language, in Figures B.1 and B.2, and reads it modulo $=_E$: a step on a term $M' =_E M$ counts as a step on M. Here its rules are restricted to the fragment of Definition 2.1, and the tables, phases and synchronisation parameters that the fragment does not use are dropped from its configurations; the communication rule, which in the manual leaves the input to match its pattern in a further step, matches the variable at once.

Rewriting. A closed destructor application $g(U_1, \ldots, U_n)$ *rewrites to* U, written $g(\vec{U}) \to U$, if $\mathrm{def}(g) = (g(U'_{i,1}, \ldots, U'_{i,n}) \to U'_i)_{i \leq k}$, j is least such that some substitution σ has $\sigma U'_{j,i} =_E U_i$ for $i = 1, \ldots, n$, and $U = \sigma U'_j$. The final rule of $\mathrm{def}(g)$ makes j exist, so every closed destructor application rewrites.

Expressions. A *term configuration* is $\mathcal{N}, D$ with $\mathcal{N}$ a finite set of names disjoint from $\mathcal{N}_{\mathrm{pub}}$ and D a closed expression with $\mathrm{fn}(D) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}$. Writing ℓ for a label that is either empty or

event e for a closed event e, the transitions $\mathcal{N}, D \xrightarrow{\ell}_t \mathcal{N}', D'$ are the least relation closed under

$$\begin{aligned}
\mathcal{N}, f(U_1, \ldots, U_n) &\to_t \mathcal{N}, \mathsf{fail} && \text{if some } U_i = \mathsf{fail} \\
\mathcal{N}, g(U_1, \ldots, U_n) &\to_t \mathcal{N}, U && \text{if } g(U_1, \ldots, U_n) \to U \\
\mathcal{N}, \mathsf{new}\ a; D &\to_t \mathcal{N} \cup \{a'\}, D\{a'/a\} && \text{if } a' \notin \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N} \\
\mathcal{N}, \mathsf{if}\ M\ \mathsf{then}\ D_1\ \mathsf{else}\ D_2 &\to_t \mathcal{N}, D_1 && \text{if } M =_E \mathsf{true} \\
\mathcal{N}, \mathsf{if}\ M\ \mathsf{then}\ D_1\ \mathsf{else}\ D_2 &\to_t \mathcal{N}, D_2 && \text{if } M \neq_E \mathsf{true} \\
\mathcal{N}, \mathsf{if\ fail\ then}\ D_1\ \mathsf{else}\ D_2 &\to_t \mathcal{N}, \mathsf{fail} \\
\mathcal{N}, \mathsf{let}\ x = M\ \mathsf{in}\ D_1\ \mathsf{else}\ D_2 &\to_t \mathcal{N}, D_1\{M/x\} \\
\mathcal{N}, \mathsf{let}\ x = \mathsf{fail}\ \mathsf{in}\ D_1\ \mathsf{else}\ D_2 &\to_t \mathcal{N}, D_2 \\
\mathcal{N}, \mathsf{event}\ ev(M_1, \ldots, M_n); D &\xrightarrow{\mathsf{event}\ ev(M_1,\ldots,M_n)}_t \mathcal{N}, D \\
\mathcal{N}, \mathsf{event}\ ev(U_1, \ldots, U_n); D &\to_t \mathcal{N}, \mathsf{fail} && \text{if some } U_i = \mathsf{fail} \\
\mathcal{N}, C[D] &\xrightarrow{\ell}_t \mathcal{N}', C[D'] && \text{if } \mathcal{N}, D \xrightarrow{\ell}_t \mathcal{N}', D',\ C \in \mathcal{C}_t,
\end{aligned}$$

where f is a constructor, g a destructor and $\mathcal{C}_t$ is the set of contexts

$$\begin{gathered}
h(U_1, \ldots, U_{i-1}, [\,], D_{i+1}, \ldots, D_n), \qquad \mathsf{if}\ [\,]\ \mathsf{then}\ D_1\ \mathsf{else}\ D_2, \\
\mathsf{let}\ x = [\,]\ \mathsf{in}\ D_1\ \mathsf{else}\ D_2, \qquad \mathsf{event}\ ev(U_1, \ldots, U_{i-1}, [\,], D_{i+1}, \ldots, D_n); D.
\end{gathered}$$

So the arguments of an application or an event are evaluated from left to right, each to a may-fail term before the next, and the evaluation goes on after an argument has failed; a may-fail term has no transition.

Processes. A *configuration* is $\mathcal{N}, \mathcal{P}$ with $\mathcal{N}$ as above and $\mathcal{P}$ a finite multiset of closed processes with $\mathrm{fn}(\mathcal{P}) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}$. The transitions $\mathcal{N}, \mathcal{P} \xrightarrow{\ell} \mathcal{N}', \mathcal{P}'$ are the least relation closed under

$$\begin{aligned}
\mathcal{N}, \mathcal{P} \cup \{\mathbf{0}\} &\to \mathcal{N}, \mathcal{P} \\
\mathcal{N}, \mathcal{P} \cup \{P \mid Q\} &\to \mathcal{N}, \mathcal{P} \cup \{P, Q\} \\
\mathcal{N}, \mathcal{P} \cup \{!P\} &\to \mathcal{N}, \mathcal{P} \cup \{P, !P\} \\
\mathcal{N}, \mathcal{P} \cup \{\mathsf{new}\ a; P\} &\to \mathcal{N} \cup \{a'\}, \mathcal{P} \cup \{P\{a'/a\}\} && \text{if } a' \notin \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N} \\
\mathcal{N}, \mathcal{P} \cup \{\mathsf{out}(N, M); Q,\ \mathsf{in}(N', x); P\} &\to \mathcal{N}, \mathcal{P} \cup \{Q,\ P\{M/x\}\} && \text{if } N =_E N' \\
\mathcal{N}, \mathcal{P} \cup \{\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q\} &\to \mathcal{N}, \mathcal{P} \cup \{P\} && \text{if } M =_E \mathsf{true} \\
\mathcal{N}, \mathcal{P} \cup \{\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q\} &\to \mathcal{N}, \mathcal{P} \cup \{Q\} && \text{if } M \neq_E \mathsf{true} \\
\mathcal{N}, \mathcal{P} \cup \{\mathsf{let}\ x = M\ \mathsf{in}\ P\ \mathsf{else}\ Q\} &\to \mathcal{N}, \mathcal{P} \cup \{P\{M/x\}\} \\
\mathcal{N}, \mathcal{P} \cup \{\mathsf{let}\ x = \mathsf{fail}\ \mathsf{in}\ P\ \mathsf{else}\ Q\} &\to \mathcal{N}, \mathcal{P} \cup \{Q\} \\
\mathcal{N}, \mathcal{P} \cup \{\mathsf{event}\ ev(M_1, \ldots, M_n); P\} &\xrightarrow{\mathsf{event}\ ev(M_1,\ldots,M_n)} \mathcal{N}, \mathcal{P} \cup \{P\} \\
\mathcal{N}, \mathcal{P} \cup \{C[D]\} &\xrightarrow{\ell} \mathcal{N}', \mathcal{P} \cup \{C[D']\} && \text{if } \mathcal{N}, D \xrightarrow{\ell}_t \mathcal{N}', D',\ C \in \mathcal{C}_p,
\end{aligned}$$

where $\mathcal{C}_p$ is the set of contexts

$$\begin{gathered}
\mathsf{out}([\,], D); P, \qquad \mathsf{out}(U, [\,]); P, \qquad \mathsf{in}([\,], x); P, \qquad \mathsf{if}\ [\,]\ \mathsf{then}\ P\ \mathsf{else}\ Q, \\
\mathsf{let}\ x = [\,]\ \mathsf{in}\ P\ \mathsf{else}\ Q, \qquad \mathsf{event}\ ev(U_1, \ldots, U_{i-1}, [\,], D_{i+1}, \ldots, D_n); P.
\end{gathered}$$

A process whose expressions have become may-fail terms with fail where a term is required, as $\mathsf{out}(\mathsf{fail}, M); P$, $\mathsf{in}(\mathsf{fail}, x); P$, if fail then P else Q or event $ev(\mathsf{fail}); P$, has no transition. A *trace* is a finite transition sequence $\mathcal{N}_0, \mathcal{P}_0 \xrightarrow{\ell_1} \ldots \xrightarrow{\ell_n} \mathcal{N}_n, \mathcal{P}_n$, written $\mathcal{N}_0, \mathcal{P}_0 \to^* \mathcal{N}_n, \mathcal{P}_n$.

Definition 2.5 (Survey, Definitions 3.1–3.5)**.** Let P_0 be the closed process of the model.

1. A closed process Q is an $\mathcal{N}_{\text{pub}}$-*adversary* if $\text{fn}(Q) \subseteq \mathcal{N}_{\text{pub}}$, every function symbol of Q is public, and Q contains no event.
2. A trace *outputs* M *publicly* if some communication step of it has the output $\mathsf{out}(N, M'); Q$ with $N \in \mathcal{N}_{\text{pub}}$ and $M' =_E M$. It *executes* event e if some step of it is labelled event e' with $e' =_E e$.
3. P_0 *preserves the secrecy of* the closed term M *from* $\mathcal{N}_{\text{pub}}$ if no trace $\mathcal{N}_{\text{priv}}, \{P_0, Q\} \to^* \mathcal{N}', \mathcal{P}'$ with Q an $\mathcal{N}_{\text{pub}}$-adversary outputs M publicly.
4. P_0 *satisfies* $\mathsf{event}(e) \rightsquigarrow \bigvee_{j=1}^{m} \bigwedge_{k=1}^{l_j} \mathsf{event}(e_{jk})$ *against* $\mathcal{N}_{\text{pub}}$-*adversaries*, for events e and e_{jk} with variables, if, for every $\mathcal{N}_{\text{pub}}$-adversary Q, every trace Tr from $\mathcal{N}_{\text{priv}}, \{P_0, Q\}$ and every substitution σ: if Tr executes event σe then there are σ' and j with $\sigma' e =_E \sigma e$ such that Tr executes event $\sigma' e_{jk}$ for every $k \le l_j$.

The survey states these for its own calculus. Here the traces are those of the manual's semantics above, an adversary may use new, if and let inside its terms, and the set of private names, which the survey chooses with the model, is $\mathcal{N}_{\text{priv}}$.

For an $\mathcal{N}_{\text{pub}}$-adversary Q write $\mathcal{C}_Q$ for the configuration $\mathcal{N}_{\text{priv}}, \{P_0, Q\}$.

Definition 2.6 (Satisfaction)**.** A trace *satisfies* $\mathsf{attacker}(M)$ if it outputs M publicly, and *satisfies* $\mathsf{event}(e)$ if it executes event e.

Definition 2.7 (Correspondences between facts)**.** Let F and H_{jk} ($j \le m$, $k \le l_j$) be query facts over X. P_0 *satisfies* $F \rightsquigarrow \bigvee_{j=1}^{m} \bigwedge_{k=1}^{l_j} H_{jk}$ *against* $\mathcal{N}_{\text{pub}}$-*adversaries* if for every $\mathcal{N}_{\text{pub}}$-adversary Q, every trace Tr from $\mathcal{C}_Q$ and every substitution σ: if Tr satisfies σF then there are σ' and j with $\sigma' F =_E \sigma F$ such that Tr satisfies $\sigma' H_{jk}$ for every $k \le l_j$. For event facts this is Definition 2.5(4).

3 The translation

The calculus is translated into the trace semantics: a destructor to a function on may-fail messages, an expression to a computation, a process to a set of traces and a query to the property it denotes. Section 3.2 relates the translation to the semantics of Section 2.

Throughout, fix a model (Definition 2.3) and instantiate the trace semantics of Section 1 from it: the signature Σ_0 has the declared constructors, the event symbols and the free names $\mathcal{N}_{\text{pub}} \uplus \mathcal{N}_{\text{priv}}$ as nullary symbols, so that $\Sigma = \Sigma_0 \uplus$ Data of Definition 1.7 has the data symbols besides and an event $ev(\vec{m})$ is a message; E is the equational theory of the model; Fr is the set of symbols of Σ_0 at the root of no equation, which contains the event symbols; Pub is the set of public declared constructors together with $\mathcal{N}_{\text{pub}}$, so that the event symbols are private; and Dt is the set of the functions of Definition 3.1 of the public declared destructors, so that the attacker has the standard operations besides. A projection then denotes the standard projection π_i^n, and a boolean destructor is not among Ops, which changes nothing, each of its results being true, false or an argument. The instantiation requires

(R1) no equation of E has a variable as a side, so that (E, Fr) satisfies (H);

(R2) in every rule of every destructor, the terms among the arguments have all their constructors in Fr.

These conditions exclude some ProVerif models: an equation with a variable as one side, such as the survey's (2.8), and a destructor whose rules match on a symbol at the root of an equation.

3.1 Destructors, expressions, processes and queries

Definition 3.1 (Destructors from rewrite rules)**.** A *valuation* θ sends ordinary variables to $\mathcal{M}$ and may-fail variables to $\mathcal{M}_\perp$, and gives a may-fail term the value $U\theta \in \mathcal{M}_\perp$: the evaluation $M\theta \in \mathcal{M}$ of a term, $\theta(u)$, and $\perp$ for fail. A destructor g with $\mathrm{def}(g) = (g(U_{i,1}, \ldots, U_{i,n}) \to U_{i,0})_{i \le k}$ denotes the function

$$g(\vec{u}) = U_{i,0}\theta \quad \text{for } i \text{ least such that some } \theta \text{ has } U_{i,j}\theta = u_j \text{ for } j = 1, \ldots, n,$$

which the final rule of $\mathrm{def}(g)$ makes total. By (R2) and Proposition 1.5 at most one θ on the variables of the i-th rule has $U_{i,j}\theta = u_j$ for all j, so g is well defined. A term among the $U_{i,j}$ thus accepts only messages, fail only $\perp$, and a may-fail variable any u_j, which is the survey's two kinds of substitution. When no rule but the final one contains fail or a may-fail variable, every other rule requires messages, so $g = g_0^{\uparrow}$ for the $g_0 : \mathcal{M}^n \to \mathcal{M}_\perp$ that those rules define on messages.

Definition 3.2 (Translation)**.** An *environment* ρ maps variables and names to $\mathcal{M}$; ρ_0 maps each free name a to the constant $a \in \mathcal{M}$. Write $\rho M \in \mathcal{M}$ for the homomorphic extension. An expression is translated to a *computation* $[\![D]\!]\rho : (\mathcal{M}_\perp \to \mathrm{Proc}) \to \mathrm{Proc}$, the process it becomes when its result is passed to a continuation κ:

$$\begin{aligned}
[\![M]\!]\rho\,\kappa &= \kappa(\rho M),\\
[\![\mathsf{fail}]\!]\rho\,\kappa &= \kappa(\perp),\\
[\![h(D_1, \ldots, D_n)]\!]\rho\,\kappa &= [\![D_1]\!]\rho\,(u_1 \mapsto [\![D_2]\!]\rho\,(u_2 \mapsto \ldots\\
&\qquad \ldots [\![D_n]\!]\rho\,(u_n \mapsto \kappa(h(u_1, \ldots, u_n)))\ldots)),\\
[\![\mathsf{new}\ a; D]\!]\rho\,\kappa &= \mathsf{new}.(m \mapsto [\![D]\!]\rho[a \mapsto m]\,\kappa),\\
[\![\mathsf{if}\ D\ \mathsf{then}\ D_1\ \mathsf{else}\ D_2]\!]\rho\,\kappa &= [\![D]\!]\rho\left(u \mapsto \begin{cases} [\![D_1]\!]\rho\,\kappa & u = \mathsf{true},\\ [\![D_2]\!]\rho\,\kappa & u \in \mathcal{M} \setminus \{\mathsf{true}\},\\ \kappa(\perp) & u = \perp, \end{cases}\right)\\
[\![\mathsf{let}\ x = D\ \mathsf{in}\ D_1\ \mathsf{else}\ D_2]\!]\rho\,\kappa &= [\![D]\!]\rho\left(u \mapsto \begin{cases} [\![D_1]\!]\rho[x \mapsto u]\,\kappa & u \in \mathcal{M},\\ [\![D_2]\!]\rho\,\kappa & u = \perp, \end{cases}\right)\\
[\![\mathsf{event}\ ev(\vec{D}); D']\!]\rho\,\kappa &= [\![D_1]\!]\rho\,(u_1 \mapsto \ldots\\
&\qquad \ldots [\![D_n]\!]\rho\,(u_n \mapsto\\
&\qquad \begin{cases} \mathsf{ev}(ev(\vec{u})).[\![D']\!]\rho\,\kappa & \vec{u} \in \mathcal{M}^n,\\ \kappa(\perp) & \text{else} \end{cases})\ldots)
\end{aligned}$$

where $h(u_1, \ldots, u_n)$ is $f^{\uparrow}(\vec{u})$ for a constructor f and the function of Definition 3.1 for a destructor

g. Processes are translated by

$$
\begin{aligned}
[\![\mathbf{0}]\!]\rho &= \mathbf{0},\\
[\![\mathsf{out}(D, D'); P]\!]\rho &= [\![D]\!]\rho\left(u \mapsto [\![D']\!]\rho\left(v \mapsto \mathsf{out}(u, v).[\![P]\!]\rho \text{ if } u, v \in \mathcal{M}, \text{ else } \mathbf{0}\right)\right),\\
[\![\mathsf{in}(D, x); P]\!]\rho &= [\![D]\!]\rho\left(u \mapsto \begin{cases} \mathsf{in}(u).\big(m \mapsto [\![P]\!]\rho[x \mapsto m]\big) & u \in \mathcal{M},\\ \mathbf{0} & u = \bot, \end{cases}\right)\\
[\![P \mid Q]\!]\rho &= [\![P]\!]\rho \mid [\![Q]\!]\rho,\\
[\![!P]\!]\rho &= ![\![P]\!]\rho,\\
[\![\mathsf{new}\ a; P]\!]\rho &= \mathsf{new}.\big(m \mapsto [\![P]\!]\rho[a \mapsto m]\big),\\
[\![\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q]\!]\rho &= [\![D]\!]\rho\left(u \mapsto \begin{cases} [\![P]\!]\rho[x \mapsto u] & u \in \mathcal{M},\\ [\![Q]\!]\rho & u = \bot, \end{cases}\right)\\
[\![\mathsf{if}\ D\ \mathsf{then}\ P\ \mathsf{else}\ Q]\!]\rho &= [\![D]\!]\rho\left(u \mapsto \begin{cases} [\![P]\!]\rho & u = \mathsf{true},\\ [\![Q]\!]\rho & u \in \mathcal{M} \setminus \{\mathsf{true}\},\\ \mathbf{0} & u = \bot, \end{cases}\right)\\
[\![\mathsf{event}\ ev(\vec{D}); P]\!]\rho &= [\![D_1]\!]\rho\left(u_1 \mapsto \ldots\right.\\
&\qquad \left.\ldots [\![D_n]\!]\rho\left(u_n \mapsto \begin{cases} \mathsf{ev}(ev(\vec{u})).[\![P]\!]\rho & \vec{u} \in \mathcal{M}^n,\\ \mathbf{0} & \text{else} \end{cases}\right)\ldots\right).
\end{aligned}
$$

The process $\mathbf{0}$ stands where the semantics of Section 2 has no transition. The translation of P_0 is $[\![P_0]\!]\rho_0$. A translated process mentions no ν_k except through new.

Remark 3.3. In the development an expression is a program in the continuation monad with answer type Proc returning a may-fail message, and a process a program in the same monad run by $\mathrm{run}(b) = b(\lambda_.\,\mathbf{0})$, with primitives $\mathrm{send}(c, m) = \lambda\kappa.\,\mathsf{out}(c, m).\kappa()$, $\mathrm{recv}(c) = \lambda\kappa.\,\mathsf{in}(c).\kappa$, $\mathrm{fresh} = \lambda\kappa.\,\mathsf{new}.\kappa$, $\mathrm{emit}(m) = \lambda\kappa.\,\mathsf{ev}(m).\kappa()$, $\mathrm{fork} = \lambda\kappa.\,\kappa(\mathrm{tt}) \mid \kappa(\mathrm{ff})$, $\mathrm{repl} = \lambda\kappa.\,!\kappa()$ and $\mathrm{stop} = \lambda_.\,\mathbf{0}$; let and if are case analysis in the metalanguage. The clauses above are the unfolding of that presentation, and the program the translator emits from a ProVerif model is that presentation.

Definition 3.4 (Translation of queries)**.** For a valuation $\sigma : X \to \mathcal{M}$ write $\sigma M \in \mathcal{M}$ for the evaluation of M with free names as constants, and $\sigma\, ev(M_1, \ldots, M_n) = ev(\sigma M_1, \ldots, \sigma M_n) \in \mathcal{M}$ for an event, and let the *instance* of a query fact under σ be $\sigma\,\mathsf{event}(e) = \mathsf{event}(\sigma e)$ and $\sigma\,\mathsf{attacker}(M) = \mathsf{knows}(\sigma M)$. The query $\mathsf{query}\ \mathsf{attacker}(M)$ denotes $\mathrm{Secret}([\![P_0]\!]\rho_0, \rho_0 M)$, and $\mathsf{query}\ F \Longrightarrow H$ denotes $\mathrm{Corr}([\![P_0]\!]\rho_0,\ \sigma \mapsto \sigma F,\ \sigma \mapsto \sigma H)$, the survey's Definition 3.5 with $=$ read as $=_E$.

3.2 The correspondence

Names of the operational semantics are the free names together with the created names; terms are interpreted in $\mathcal{M}$ through a bijection ι between the names outside $\mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$, those that new may create, and $\mathbb{N}$, extended by $\iota a = a$ for free names, $\iota a' = \nu_{\iota a'}$ for created names, $\iota f(\vec{M}) = f(\iota\vec{M})$, $\iota\, ev(\vec{M}) = ev(\iota\vec{M})$ for an event and $\iota\,\mathsf{fail} = \bot$; $\iota M = \iota M'$ exactly when $M =_E M'$, and likewise for events. Let ρ_ι be the environment with $\rho_\iota(a) = \iota a$ for every free or created name a.

Lemma 3.5 (Evaluation). *Let D be a closed expression over free and created names without* new, if, let *and* event. *Then $\mathcal{N}, D \to_t^* \mathcal{N}, U$ for some may-fail term U, and for every such U, $[\![D]\!]\rho_\iota = \kappa \mapsto \kappa(\iota U)$.*

Definition 3.6 (Labelled semantics). A *labelled term configuration* is (U, D) and a *labelled configuration* is $(U, \mathcal{P})$, with $U \subseteq \mathbb{N}$ finite, D a closed expression and $\mathcal{P}$ a finite multiset of closed processes, whose names are free names and symbols ν_k. The transitions of Section 2 become labelled steps, with $=_E$ read on $\mathcal{M}$: new a creates ν_k for any $k \notin U$ and adds k to U, with the label new(k); the steps on event e have the label ev$([e])$, the event read as a message; the communication has the label comm$([N], [M])$; the context rules carry the label of the step they lift; and every other step is a τ-step. There are besides the *offers*

$$(U, \mathcal{P} \cup \{\mathsf{out}(N, M); P\}) \xrightarrow{\mathsf{out}([N],[M])} (U, \mathcal{P} \cup \{P\}),$$
$$(U, \mathcal{P} \cup \{\mathsf{in}(N, x); P\}) \xrightarrow{\mathsf{in}([N],[M])} (U, \mathcal{P} \cup \{P\{M/x\}\}) \quad \text{for every closed term } M.$$

The communication is visible rather than a τ-step because a trace records a completed communication, which is what known reads the attacker's knowledge off. With $\overset{\alpha}{\Longrightarrow} = \overset{\tau}{\to}^* \overset{\alpha}{\to} \overset{\tau}{\to}^*$, the set of *weak traces* is

$$\mathrm{Tr}(U, \mathcal{P}) = \{\alpha_1 \cdots \alpha_n \in \mathrm{Act}^* \mid (U, \mathcal{P}) \overset{\alpha_1}{\Longrightarrow} \cdots \overset{\alpha_n}{\Longrightarrow}\},$$

and $(U, D) \overset{t}{\Rightarrow} (U', D')$ means that the term configuration (U, D) reaches (U', D') by steps whose visible labels, in order, are t.

Lemma 3.7 (Expressions). *Let D be a closed expression whose names are free names and symbols ν_k, let $U \subseteq \mathbb{N}$ be finite, let $\kappa : \mathcal{M}_\perp \to \mathrm{Proc}$ satisfy $\varepsilon \in \kappa(u)$ for every u, and let $t \in \mathrm{Act}^*$ have no letter of U in* nonces(t). *Then $t \in [\![D]\!]\rho\,\kappa$, where ρ extends ρ_0 by $\rho(\nu_k) = \nu_k$, if and only if $t = t_1 t_2$ for some $(U, D) \overset{t_1}{\Longrightarrow} (U', D')$ such that either $t_2 = \varepsilon$, or D' is a may-fail term, $t_2 \in \kappa(\iota D')$ and no letter of U' is in* nonces(t_2).

Theorem 3.8 (Adequacy). *For every closed process P whose names are free names and symbols ν_k, $[\![P]\!]\rho = \mathrm{Tr}(\emptyset, \{P\})$, where ρ extends ρ_0 by $\rho(\nu_k) = \nu_k$. For all finite multisets $\mathcal{P}, \mathcal{Q}$ of such processes,* $\mathrm{Tr}(\emptyset, \mathcal{P} \uplus \mathcal{Q}) = \mathrm{Tr}(\emptyset, \mathcal{P}) \mid \mathrm{Tr}(\emptyset, \mathcal{Q})$.

Let $\mathcal{A}_0 = \mathrm{Attacks}([\![P_0]\!]\rho_0)$. For a trace Tr from $\mathcal{C}_Q$ let outputs(Tr) $\subseteq \mathcal{M}$ be the set of ιM for the terms M that Tr outputs publicly, and for $t \in \mathrm{Act}^*$ say that Tr and t *have the same events* if Tr executes event e exactly when ev(ιe) occurs in t.

Theorem 3.9 (Attacks). *Fix an $\mathcal{N}_{\mathrm{pub}}$-adversary Q and a trace* Tr *from $\mathcal{C}_Q$, as well as a trace $t \in \mathcal{A}_0$.*

(E) (i) There is $t' \in \mathcal{A}_0$ with the same events as Tr.

(ii) There are an $\mathcal{N}_{\mathrm{pub}}$-adversary Q' and a trace Tr$'$ *from $\mathcal{C}_{Q'}$ with the same events as t.*

(K) Suppose further $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$.

(i) For every $m \in$ outputs(Tr) *there is t' as in (E)(i) such that $m \in \mathrm{known}(\langle\emptyset\rangle, t')$.*

(ii) For every $m \in \mathrm{known}(\langle\emptyset\rangle, t)$ there are Q' and Tr$'$ *as in (E)(ii) such that $m \in$* outputs(Tr$'$).

Theorem 3.10 (Secrecy)**.** *Let* $\mathcal{N}_{\text{pub}} \neq \emptyset$*, let* M *be a closed term with* $\text{fn}(M) \subseteq \mathcal{N}_{\text{pub}} \cup \mathcal{N}_{\text{priv}}$ *and let* $s = \rho_0 M$*. Then* $\text{Secret}([\![P_0]\!]\rho_0, s)$ *holds if and only if* P_0 *preserves the secrecy of* M *from* $\mathcal{N}_{\text{pub}}$*.*

Remark 3.11. The hypothesis $\mathcal{N}_{\text{pub}} \neq \emptyset$ cannot be dropped: when $\mathcal{N}_{\text{pub}} = \emptyset$ no trace outputs anything publicly, so P_0 preserves the secrecy of every M, whereas $\text{Secret}([\![P_0]\!]\rho_0, \rho_0 M)$ fails whenever $\emptyset \vdash \rho_0 M$, for instance for a public constant M.

Theorem 3.12 (Correspondences)**.** *Let* F *and* H *be query facts over* X *such that* H *is an event fact, and let* $\mathcal{N}_{\text{pub}} \neq \emptyset$ *if* F *is an attacker fact. Then* P_0 *satisfies* $F \rightsquigarrow H$ *against* $\mathcal{N}_{\text{pub}}$*-adversaries if and only if* $\text{Corr}([\![P_0]\!]\rho_0,\ \sigma \mapsto \sigma F,\ \sigma \mapsto \sigma H)$*.*

Remark 3.13. Neither hypothesis can be dropped. When $\mathcal{N}_{\text{pub}} = \emptyset$ and $F = \mathsf{attacker}(M)$, P_0 satisfies the correspondence vacuously, whereas Corr fails as soon as $\emptyset \vdash \sigma M$ for some valuation σ. For $H = \mathsf{attacker}(M)$, Definition 2.6 requires σM itself to be output publicly, whereas $\mathsf{knows}(\sigma M)$ holds as soon as σM is derivable: with the symmetric encryption of the survey's §2.1, $\mathcal{N}_{\text{pub}} = \{c\}$, an event symbol e of arity 0 and $P_0 = \mathsf{out}(c, \mathsf{senc}(s, k)); \mathsf{out}(c, k); \mathsf{event}\ e$, Corr holds for $\mathsf{event}(e) \rightsquigarrow \mathsf{attacker}(s)$ while the correspondence of Definition 2.7 fails. The reading by derivability is ProVerif's.

References

[1] Bruno Blanchet. Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1–2):1–135, October 2016. `../Reference/BlanchetFnTPS16.pdf`, doi:10.1561/3300000004.

[2] Bruno Blanchet, Ben Smyth, Vincent Cheval, and Marc Sylvestre. ProVerif 2.05: Automatic cryptographic protocol verifier, user manual and tutorial. October 2023. `../Reference/proverif-manual.pdf`.